

# Module 5b Exercises

## Efficient SQL in Yellowbrick — Performance Investigation

|                      |                                                                                  |
|----------------------|----------------------------------------------------------------------------------|
| <b>Environment</b>   | Yellowbrick Cloud Test Environment — version 7.4.3                               |
| <b>Tools</b>         | Yellowbrick Manager (SQL Editor) or DBeaver                                      |
| <b>Sample data</b>   | samples.retail — read-only, shared by all students (351.5M-row fact, 4 nodes)    |
| <b>Your database</b> | db_<username> (the personal database you created in Module 3)                    |
| <b>Schema prefix</b> | Read from samples.retail.<table>; build all helper tables in your own database   |
| <b>Goal</b>          | Find a performance anti-pattern with EXPLAIN, fix it, and measure the difference |

✓ **How this module works** These exercises are an investigation, not a quiz. Each one gives you a query that performs **badly**, shows you how to **see** why in the query plan, then has you **fix** it and **measure** the result. Every query here was run on this exact cluster — the plan shapes and numbers are real.

## Setup — Do This First

All read queries below run against the shared, read-only samples.retail schema. Several exercises ask you to build small helper tables (copies of the fact distributed or sorted differently). Build those in your own database, db\_<username>, so everyone has an isolated workspace.

 **Yellowbrick Manager** Open the SQL Editor and confirm your context. If you have not created a personal database yet, follow Module 3 §2.1 first.

### S.1 Confirm your context and the sample data

```
SELECT current_database() AS my_db, current_user AS me;

-- The shared fact: 351,556,736 rows, hashed on its surrogate PK (sales_key),
-- with NO sort key. Those two facts are what make it a good teaching set.
SELECT count(*) FROM samples.retail.fact_sales; -- 351,556,736
```

### S.2 The query-tag + stats helper (build once)

Every exercise follows the same loop: tag a query, run it, then read its real metrics from the system logs. To make that easy, create this small stored procedure once in your own database. Tag each query with an inline comment such as /\* ex1\_bad \*/ — in YB Manager every Run is a new session, and the comment travels with the query text into sys.log\_query, so the lookup works regardless of session.

```
-- 1. A result type for the procedure to return
```

```

DROP TABLE IF EXISTS qstat_rs;
CREATE TABLE qstat_rs (
  tag VARCHAR(64), query_id BIGINT,
  run_ms NUMERIC(12,0),    -- pure execution time
  total_ms NUMERIC(12,0),  -- incl. compile + queue
  read_mb NUMERIC(12,1),   -- bytes read from storage
  net_mb NUMERIC(12,1),    -- bytes redistributed across workers
  spill_mb NUMERIC(12,1),  -- bytes spilled to disk
  rows_out BIGINT);

-- 2. Pull the most recent runs whose query text contains your tag
CREATE OR REPLACE PROCEDURE show_query_stats(p_tag VARCHAR)
RETURNS SETOF qstat_rs
LANGUAGE plpgsql AS $$
DECLARE r qstat_rs%ROWTYPE;
BEGIN
  FOR r IN
    SELECT SUBSTRING(query_text FROM '/*[*] *([a-zA-Z0-9_]+) *[*]/*'),
           query_id, ROUND(run_ms,0), ROUND(total_ms,0),
           ROUND(io_read_bytes / 1024.0^2, 1),
           ROUND(io_network_bytes / 1024.0^2, 1),
           ROUND(io_spill_space_bytes_max / 1024.0^2, 1),
           rows_returned
    FROM sys.log_query
    WHERE username = current_user          -- only YOUR runs (multi-user safe)
          AND query_text LIKE '%' || p_tag || '%'
          AND query_text NOT LIKE '%sys.log_query%'
          AND type IN ('select','ctas','insert','update','delete')
    ORDER BY submit_time DESC LIMIT 20
  LOOP RETURN NEXT r; END LOOP;
END; $$;

```

Using it — tag the original and the rewrite, run both, then compare:

```

/* ex1_bad */ SELECT count(*) FROM samples.retail.fact_sales WHERE date_key::VARCHAR LIKE
'2025%';
/* ex1_good */ SELECT count(*) FROM samples.retail.fact_sales WHERE date_key BETWEEN
20250101 AND 20251231;

SELECT * FROM show_query_stats('ex1');    -- both rows, newest first

```

 **Tip** Prefer the inline `/* tag */` comment over `SET ybd_query_tags`. The comment is recorded in `query_text` regardless of session. If you keep one session open (e.g. in DBeaver), `SET ybd_query_tags TO 'ex1_bad'`; also works and fills the dedicated `tags` column.

🕒 **How to read a plan** Reading the plan matters more than any single number. On every `EXPLAIN (ANALYZE)` read two things: (1) `rows_planned` vs `rows_actual` — a large gap means the optimizer was misled; and (2) any `DISTRIBUTE ON HASH(...)` node — that is data moving between workers.

This cluster has 4 worker nodes, so redistribution is real work. In the `EXPLAIN` footer the `Distributed` figure (e.g. `Distributed: 245.94MiB` in Exercise 13) shows how much data crossed workers; the `Network` field frequently still reads `0.00B` on this build, so judge redistribution by the `DISTRIBUTE ON HASH` node, the rows flowing through it, and the `Distributed` bytes — not by `net_mb` alone. Yellowbrick also pre-aggregates

locally (GROUP BY PARTIAL) before shuffling, so a heavy GROUP BY often moves only a few thousand rows even over 351M.

### S.3 The samples.retail facts you will rely on

Confirmed on this cluster (your numbers should match):

| Table        | Rows        | Distribution              | Notes                                            |
|--------------|-------------|---------------------------|--------------------------------------------------|
| fact_sales   | 351,556,736 | HASH (sales_key), no SORT | date_key 20250515–20260516; txn_id 1.0M distinct |
| dim_customer | 99,994      | HASH (customer_key)       | membership_tier: 4 values                        |
| dim_product  | 50,000      | HASH (product_key)        | category: 20 values; brand: 40                   |
| dim_store    | 100         | HASH (store_key)          | city, state, country                             |
| dim_date     | 367         | HASH (date_key)           | full_date, year (2025–2026), month, is_weekend   |

✓ **Why this schema** Two design quirks make this a great teaching set: (1) the fact is distributed on its **surrogate PK** (sales\_key), so it is **not collocated with any dimension** — every star join must move the dimension to the fact; and (2) the fact has **no sort key**, so date scans cannot skip blocks until you build a sorted copy. Unlike some sample sets, the fact keys here are clean subsets of the dimensions, so inner joins lose no rows.

## Exercise Overview

| Exercise | What you will do                                        | Time    |
|----------|---------------------------------------------------------|---------|
| 1        | Filtering — keep the column bare so the scan can prune  | ~10 min |
| 2        | Joins — collocation, broadcast, and join elimination    | ~10 min |
| 3        | Joins — implicit casts & expressions in the ON clause   | ~7 min  |
| 4        | Joins — controlling order with a CTE + OFFSET 0         | ~7 min  |
| 5        | Aggregation — GROUP BY on the distribution column       | ~7 min  |
| 6        | Aggregation — COUNT(DISTINCT) on a columnar engine      | ~8 min  |
| 7        | Materialization — CTAS/TEMP for stats and the right key | ~7 min  |
| 8        | DML — singleton vs set-based; recreate vs large delete  | ~7 min  |
| 9        | Stored procedures — push the predicate into the backend | ~6 min  |

|    |                                                  |        |
|----|--------------------------------------------------|--------|
| 10 | Data types — functions on hot columns; JSON cost | ~6 min |
| 11 | Views — flatten a nested view for pushdown       | ~6 min |
| 12 | Planner hinting — raising join_collapse_limit    | ~6 min |

## Exercise 1 — Filtering: Keep the Column Bare

Time: ~10 minutes

- **Concept** To skip blocks, a predicate must compare a **bare column** to a constant. Wrap the column in a function or a cast and Yellowbrick can no longer use the column's min/max metadata — it scans everything, and its row estimate collapses.

Optional helper — a copy sorted on date\_key, so range filters can actually skip blocks:

```
DROP TABLE IF EXISTS my_fact_dt;
CREATE TABLE my_fact_dt AS
  SELECT * FROM samples.retail.fact_sales
  DISTRIBUTE ON (sales_key)
  SORT ON (date_key);
```

- ⚠ **Offending pattern** Both queries below put an expression on the scanned column, so the scan cannot prune:

```
/* ex1_bad */ SELECT count(*) FROM samples.retail.fact_sales
               WHERE date_key::VARCHAR LIKE '2025%';      -- cast on the column

/* ex1_bad2 */ SELECT count(*) FROM samples.retail.fact_sales
               WHERE LOWER(discount_type) = 'percentage'; -- function on the column
```

- 📺 **Find it** Run `EXPLAIN ANALYZE` on `ex1_bad`. The planner rewrites your filter to `LEFT(date_key::VARCHAR(256),4) = '2025'` — there is **no** `scan_constraints` line, and **rows\_planned is 1,757,784 while rows\_actual is 221,559,040** — the estimate is off by ~126x. That bad estimate is what wrecks downstream joins.

- ✓ **Fix** Keep the column bare and move the work to the constant side. `date_key` is an integer in `YYYYMMDD` form, so a numeric range is exact:

```
/* ex1_good */ SELECT count(*) FROM samples.retail.fact_sales
                WHERE date_key BETWEEN 20250101 AND 20251231;

/* ex1_good2 */ SELECT count(*) FROM samples.retail.fact_sales
                WHERE discount_type = 'percentage';
```

🔍 **Verify** On `ex1_good` the plan now shows `scan_constraints: min_max(date_key)` and **rows\_planned = 221,503,644** — within 0.03% of the 221,559,040 actual. On the unsorted base table the bytes read are similar, but the bare-column filter is already faster (it skips the per-row cast) and keeps the estimate exact. Re-run the same range against `my_fact_dt` (sorted on `date_key`) and the scan also skips blocks, so read and run\_ms drop further. That block-skipping is what the sort key buys you.

🔍 **What you should see** `ex1_bad`: rows\_planned 1,757,784 vs actual 221,559,040, no pruning, 1.63 GiB read, ~1.5 s. `ex1_good`: rows\_planned 221,503,644 (exact), min/max pruning present, 1.77 GiB read, ~0.85 s — faster and correctly estimated. On `my_fact_dt` the same filter reads far fewer blocks again.

🎯 **Your turn** Write a filter that counts only December-2025 sales using a bare-column `date_key` range (*hint: 20251201 to 20251231*). Confirm the plan shows `scan_constraints` and an exact estimate.

## Exercise 2 — Joins: Collocation, Broadcast, and Join Elimination

Time: ~10 minutes

● **Concept** Matching rows must sit on the same worker. A small dimension is cheaply **broadcast** (replicated) to every worker. The expensive case is redistributing two **large** tables joined on a non-distribution key.

### Part A — what the planner does with a small dimension

```
/* ex2a */ EXPLAIN
SELECT c.membership_tier, count(*)
FROM   samples.retail.fact_sales f
JOIN   samples.retail.dim_customer c ON f.customer_key = c.customer_key
GROUP BY c.membership_tier;
```

📺 **Find it** In the plan the 99,994-row `dim_customer` is the BUILD side with `DISTRIBUTE REPLICATE` — broadcast to every worker (cheap) — while `fact_sales` is scanned in place behind a bloom filter. The planner already collocates small dims for you; declaring them `DISTRIBUTE REPLICATE` in DDL just makes it explicit.

🖋️ **Tip** Now change the SELECT list to just `count (*)` and re-EXPLAIN: the join **disappears** entirely. With no dimension columns needed and the foreign key satisfied, the planner eliminates the join and scans only the fact. Because the keys here are clean, the eliminated count equals the joined count — the optimization is correct, and free.

## Part B — the redistribution you can design away

Build a per-transaction table plus two fact copies on different keys, and refresh stats:

```
DROP TABLE IF EXISTS my_txn, my_sales_bad, my_sales_good;
CREATE TABLE my_txn AS
  SELECT transaction_id, SUM(total) AS txn_total
  FROM samples.retail.fact_sales GROUP BY transaction_id  -- ~1.0M rows
  DISTRIBUTE ON (transaction_id);
CREATE TABLE my_sales_bad AS SELECT * FROM samples.retail.fact_sales DISTRIBUTE ON
(sales_key);
CREATE TABLE my_sales_good AS SELECT * FROM samples.retail.fact_sales DISTRIBUTE ON
(transaction_id);
ANALYZE my_txn; ANALYZE my_sales_bad; ANALYZE my_sales_good;  -- fresh stats!
```

```
/* ex2_bad */ SELECT count(*) FROM my_sales_bad s JOIN my_txn t ON s.transaction_id =
t.transaction_id;
/* ex2_good */ SELECT count(*) FROM my_sales_good s JOIN my_txn t ON s.transaction_id =
t.transaction_id;
```



**Find it** EXPLAIN (ANALYZE) both. In the classic case the 351M side would carry a `DISTRIBUTE ON HASH(transaction_id)` node in `ex2_bad` and none in `ex2_good`. On this cluster watch what the planner actually does with `my_txn` (see the note below) — the signal to learn is the `DISTRIBUTE` node over the large table, and whether both tables share the key so it can be avoided.

🔍 **Verify** `ex2_good` removes the reshuffle of the large table.

✓ **Expect on this cluster** Even at 351M rows the planner decides the ~1.0M-row `my_txn` is small enough to **broadcast** (`DISTRIBUTE REPLICATE`) in both variants, scanning the fact in place behind a bloom filter — so both run in the same ballpark (the join reads ~3.45 GiB and takes ~10 s). That is itself the Part A lesson: a 1M-row build side fits the broadcast threshold. The redistribution you can clearly see and design away is the `GROUP BY` in Exercise 5. Always **ANALYZE after a CTAS**: stale stats are what push the planner into a needless reshuffle.

## Exercise 3 — Joins: Implicit Casts & Expressions in ON

Time: ~7 minutes

● **Concept** Mismatched key types or functions in the `ON` clause inject a per-row cast on every row of the large side, and usually a redistribution plus a bad estimate.

The integer keys in `samples.retail` already match, so build a realistic source feed with a mismatched key type:

```
DROP TABLE IF EXISTS my_feed;
CREATE TABLE my_feed (
  customer_key BIGINT,          -- INT8 vs dim_customer.customer_key INT4
  amount NUMERIC(12,2)
) DISTRIBUTED ON (customer_key);
INSERT INTO my_feed
  SELECT customer_key, SUM(total) FROM samples.retail.fact_sales GROUP BY customer_key;
```

**⚠ Offending pattern** A type mismatch, and a function on both sides of a text key:

```
/* ex3_bad */ SELECT count(*) FROM my_feed f
               JOIN samples.retail.dim_customer c ON f.customer_key = c.customer_key;

/* ex3_bad2 */ SELECT count(*)
               FROM samples.retail.dim_customer a
               JOIN samples.retail.dim_customer b ON UPPER(a.customer_id) =
               UPPER(b.customer_id);
```

**▶ Find it** EXPLAIN ex3\_bad shows the injected cast in the join condition: INNER HASH JOIN ON (c.customer\_key::INT8 = f.customer\_key::INT8), plus a calculate: f.customer\_key::INT8 on the fact scan — that cast runs on every row. ex3\_bad2 shows calculate: UPPER(...) on both scans for the same reason.

**✓ Fix** Match the types (customer\_key INTEGER in my\_feed), or materialize the expression as a stored column (customer\_id\_uc = UPPER(customer\_id)) and join the bare column.

**🕒 Verify** No :: cast and no calculate: on the scan in the rewritten plan; estimates tighten.

## Exercise 4 — Joins: Controlling Order with CTE + OFFSET 0

Time: ~7 minutes

**● Concept** The planner only permutes the first 8 tables, and only as well as its estimates allow. When a row-reducing filter is buried, pin it first with a CTE + OFFSET 0.

**⚠ Offending pattern** A wide join where the selective date filter is not obviously applied first:

```
/* ex4_bad */ SELECT count(*)
FROM samples.retail.fact_sales f
JOIN samples.retail.dim_customer c ON f.customer_key = c.customer_key
JOIN samples.retail.dim_product  p ON f.product_key  = p.product_key
JOIN samples.retail.dim_date      d ON f.date_key     = d.date_key
WHERE d.year = 2026;
```

✓ **Fix** Force the selective fact-to-date join to build first:

```
/* ex4_good */
WITH fd AS (
  SELECT f.* FROM samples.retail.fact_sales f
  JOIN samples.retail.dim_date d ON f.date_key = d.date_key
  WHERE d.year = 2026
  OFFSET 0 -- force this to materialize first
)
SELECT count(*)
FROM fd
JOIN samples.retail.dim_customer c ON fd.customer_key = c.customer_key
JOIN samples.retail.dim_product p ON fd.product_key = p.product_key;
```

▶ **Find it** Compare join order and peak intermediate `rows_actual` in EXPLAIN (ANALYZE) before vs after.

🕒 **What you should see** With only four tables and good stats the planner already pushes the date join first: the 136 matching `dim_date` rows are broadcast and the fact is bloom-filtered down to ~130.0M rows (year 2026) before the other joins — Read 2.56 GiB, ~1.5 s. The CTE + OFFSET 0 lever is the one to reach for when you have **more than 8 tables** or visibly wrong estimates.

## Exercise 5 — Aggregation: GROUP BY on the Distribution Column

Time: ~7 minutes

● **Concept** Grouping on a **non-distribution** column forces Yellowbrick to redistribute every row first so that all rows of a group land together. Group on the distribution column and the aggregation is local.

Build a copy of the fact distributed on `store_key`:

```
DROP TABLE IF EXISTS my_fact_bystore;
CREATE TABLE my_fact_bystore AS
  SELECT * FROM samples.retail.fact_sales DISTRIBUTE ON (store_key);
```

```
/* ex5_bad */ -- group on a NON-distribution column -> shuffle
SELECT count(*) FROM (SELECT customer_key FROM my_fact_bystore GROUP BY 1) q;

/* ex5_good */ -- group on the distribution column -> local
SELECT count(*) FROM (SELECT store_key FROM my_fact_bystore GROUP BY 1) q;
```

 **Find it** EXPLAIN (ANALYZE) ex5\_bad shows a `DISTRIBUTE ON HASH(customer_key)` node under the aggregate — every row is reshuffled. ex5\_good aggregates locally with no such node. (You can see the same effect directly on `samples.retail.fact_sales`, which is hashed on `sales_key`: a `GROUP BY customer_key` there redistributes; a `GROUP BY` on the distribution key would not.)

 **Verify** The distribution-column group has no shuffle and a lower `run_ms`.

 **Your turn** Pick a table's distribution key to match its heaviest `GROUP BY` or join. If reports group by `store_key`, distribute the table on `store_key`.

## Exercise 6 — Aggregation: COUNT(DISTINCT) on a Columnar Engine

*Time: ~8 minutes*

 **Concept** Each `COUNT(DISTINCT)` has to deduplicate its column across all workers, so several at once cost more than one. But because Yellowbrick is **columnar**, each distinct reads only the columns it needs — which changes which "fixes" actually help.

### Part A — one distinct: the engine already rewrites it

```
/* ex6_one */ SELECT store_key, COUNT(DISTINCT transaction_id) AS dc
               FROM samples.retail.fact_sales GROUP BY store_key;
```

 **Find it** Read the plan. YB 7.4 internally turns a single `COUNT(DISTINCT)` into a two-level grouping: `GROUP BY (store_key, transaction_id) then GROUP BY (store_key)`. So hand-rewriting a **single** distinct buys nothing here — confirm that for yourself.

### Part B — three distincts: each gets its own scan

```
/* ex6_three */
SELECT store_key,
       COUNT(DISTINCT customer_key),
       COUNT(DISTINCT product_key),
       COUNT(DISTINCT transaction_id)
FROM samples.retail.fact_sales GROUP BY store_key;
```

 **Find it** The plan now has **three separate SCAN fact\_sales nodes** joined back together on `store_key` — one per distinct. But each scan is columnar: it reads only **store\_key + its one key column**, not the whole 27-column row.

🕒 **What you should see** Measured at 351M rows — the three single distincts read store\_key + one key each: customer\_key 1.49 GiB / ~2.1 s, product\_key 1.16 GiB / ~1.0 s, transaction\_id 2.02 GiB / ~6.0 s. The combined three-distinct query reads **4.67 GiB in ~12.2 s** — exactly the sum (1.49 + 1.16 + 2.02). There is no wide-row penalty to remove: columnar projection already reads only the columns each distinct touches.

## Part C — the row-store "fix" that does NOT help here

⚠️ **Offending pattern** Coming from a row-store, the instinct is to materialize the base columns **once** and count distinct from that copy — the idea being to avoid re-reading a wide table three times:

```
/* ex6_fix_rowstore */ -- looks helpful, but is slower on Yellowbrick
DROP TABLE IF EXISTS base;
CREATE TEMP TABLE base AS
  SELECT store_key, customer_key, product_key, transaction_id
  FROM samples.retail.fact_sales
  DISTRIBUTE ON (store_key);
SELECT store_key,
       COUNT(DISTINCT customer_key),
       COUNT(DISTINCT product_key),
       COUNT(DISTINCT transaction_id)
FROM base GROUP BY store_key;
```

🕒 **Verify** It is **slower**, not faster. The three distincts already read only 4.67 GiB total because of columnar projection — there is no wide-row cost to eliminate. The rewrite **adds** a full 351M-row write to build "base", then re-reads the same columns three times — strictly more I/O. "Materialize once" is a row-store optimization (where every scan reads full rows); a columnar engine already does the projection for free.

✓ **When it does help** When materializing the source **does** pay off: when the distincts read from an expensive **non-materialized CTE or a nested view** that would otherwise be recomputed per distinct, or a memory-tight plan that spills — then computing it once into a TEMP table avoids the repeated work. That is the lesson of Exercise 7, not a property of base-table distinct counts. For exact distinct counts on a base table, the three-scan plan is already close to optimal; the only real lever is to compute **fewer** distincts, or relax to an approximate count when the engine supports it.

## Exercise 7 — Materialization: CTAS/TEMP for Stats and the Right Key

Time: ~7 minutes

● **Concept** A subquery or CTE is **not** materialized and carries **no statistics**, so the planner guesses

and may recompute it. A TEMP/CTAS table gives you real stats plus the distribution you choose, computed once.

⚠ **Offending pattern** An inline per-customer aggregate joined back to the dimension:

```
/* ex7_bad */
SELECT c.membership_tier, AVG(a.spend)
FROM samples.retail.dim_customer c
JOIN ( SELECT customer_key, SUM(total) AS spend
      FROM samples.retail.fact_sales GROUP BY customer_key ) a
  ON c.customer_key = a.customer_key
GROUP BY c.membership_tier;
```

✓ **Fix** Materialize the aggregate with the join key as its distribution key:

```
/* ex7_good */
DROP TABLE IF EXISTS cust_spend;
CREATE TEMP TABLE cust_spend AS
  SELECT customer_key, SUM(total) AS spend
  FROM samples.retail.fact_sales GROUP BY customer_key
  DISTRIBUTE ON (customer_key);      -- right key + statistics now exist
ANALYZE cust_spend;
SELECT c.membership_tier, AVG(s.spend)
FROM samples.retail.dim_customer c
JOIN cust_spend s ON c.customer_key = s.customer_key
GROUP BY c.membership_tier;
```

🔍 **Verify** Compare `rows_planned` vs `rows_actual` on the join — once `cust_spend` has stats and shares the `customer_key` distribution, the estimate is exact and any redistribution under the join is gone.

## Exercise 8 — DML: Set-Based Beats Singleton; Recreate Beats Big Delete

*Time: ~7 minutes (take-home)*

● **Concept** Per-row UPDATE/DELETE and large in-place modifications are expensive and leave behind delete-tracking. Stage the change and apply it in **one** statement, or rebuild the table with CTAS.

Build a working copy and a small changes table:

```
DROP TABLE IF EXISTS my_sales, my_changes;
CREATE TABLE my_sales AS SELECT * FROM samples.retail.fact_sales DISTRIBUTE ON
(sales_key);
CREATE TABLE my_changes AS                -- ~100k rows to bump 5%
```

```
SELECT sales_key, total*1.05 AS new_total
FROM samples.retail.fact_sales WHERE discount_amount > 0 LIMIT 100000;
```

⚠ **Offending pattern** One row at a time, and a large in-place delete:

```
-- (simulated) one-at-a-time, run many times
UPDATE my_sales SET total = :v WHERE sales_key = :one_key;
-- large in-place delete
DELETE FROM my_sales WHERE date_key < 20251001;
```

✓ **Fix** Set-based update, and recreate instead of a big delete:

```
/* ex8_setbased */
UPDATE my_sales t SET total = c.new_total
FROM my_changes c WHERE t.sales_key = c.sales_key;      -- one statement

/* ex8_recreate */ -- recreate instead of large delete
DROP TABLE IF EXISTS my_sales_new;
CREATE TABLE my_sales_new AS
  SELECT * FROM my_sales WHERE date_key >= 20251001
  DISTRIBUTE ON (sales_key) SORT ON (date_key);
-- then DROP my_sales; and rename my_sales_new.
```

🔍 **Verify** Watch `delete_info_bytes` for `my_sales` in `sys.table` grow under the in-place delete; the CTAS recreate leaves none. Compare total `run_ms`. Use `TRUNCATE` or `DROP` to discard a whole table cheaply rather than `DELETE`.

## Exercise 9 — Stored Procedures: Push the Predicate into the Backend

Time: ~6 minutes (take-home)

● **Concept** Filtering a procedure's output runs on the manager **after** the whole set has been built. Push the filter into the backend query instead.

```
DROP TABLE IF EXISTS store_rev_rs;
CREATE TABLE store_rev_rs (store_key INT, revenue NUMERIC(18,2));
CREATE OR REPLACE PROCEDURE store_rev() RETURNS SETOF store_rev_rs
LANGUAGE plpgsql AS $$
DECLARE r store_rev_rs%ROWTYPE;
BEGIN
  FOR r IN SELECT store_key, SUM(total)
            FROM samples.retail.fact_sales GROUP BY store_key
  LOOP RETURN NEXT r; END LOOP;
END; $$;
```

⚠ **Offending pattern** Filter the output — the whole set materializes on the manager first:

```
/* ex9_bad */ SELECT * FROM store_rev() WHERE store_key = 42;
```

✓ **Fix** Add an argument and filter inside the backend query:

```
CREATE OR REPLACE PROCEDURE store_rev(a_store INT) RETURNS SETOF store_rev_rs
LANGUAGE plpgsql AS $$
DECLARE r store_rev_rs%ROWTYPE;
BEGIN
  FOR r IN SELECT store_key, SUM(total)
            FROM samples.retail.fact_sales
            WHERE store_key = a_store          -- pushed into the scan
            GROUP BY store_key
  LOOP RETURN NEXT r; END LOOP;
END; $$;
/* ex9_good */ SELECT * FROM store_rev(42);
```

🔍 **Verify** In `sys.log_query` the embedded backend query returns the whole set in the bad case (all 100 stores aggregated) and a single group with the argument.

## Exercise 10 — Data Types: Functions on Hot Columns; JSON Cost

Time: ~6 minutes (take-home)

● **Concept** Expressions on hot join/filter columns disable skipping and estimates (Exercises 1 and 3). JSON and regex functions add a parse cost on **every row**.

⚠ **Offending pattern** A function on the filter column forces a full scan:

```
/* ex10_bad */ SELECT count(*) FROM samples.retail.fact_sales
WHERE UPPER(discount_type) = 'PERCENTAGE';
```

✓ **Fix** Filter the bare column:

```
/* ex10_good */ SELECT count(*) FROM samples.retail.fact_sales
WHERE discount_type = 'percentage';
```

 **Find it** EXPLAIN shows `calculate: UPPER(discount_type)` on the bad scan and no usable `scan_constraints`; the bare-column version compares directly.

✓ **On JSON** JSON: `samples.retail.staging` holds one semi-structured `data` column (the raw feed the star was built from). Per-row extraction with `json_extract_path_text(...)` is expensive at scale. The fix is to extract each field into a typed column **once at load** (store as JSONB if you query it repeatedly), then filter and aggregate the materialized column — which is exactly how `fact_sales` was built from staging.

## Exercise 11 — Views: Flatten a Nested View

*Time: ~6 minutes (take-home)*

● **Concept** Nested views rebuild their join order from the inside out and can block predicate pushdown, so a filter on the outer view runs late.

Build a two-level view over the star:

```
DROP VIEW IF EXISTS v_sales_top;
DROP VIEW IF EXISTS v_sales_enriched;
CREATE VIEW v_sales_enriched AS
  SELECT f.*, c.membership_tier, p.category, d.year
  FROM samples.retail.fact_sales f
  JOIN samples.retail.dim_customer c ON f.customer_key = c.customer_key
  JOIN samples.retail.dim_product p ON f.product_key = p.product_key
  JOIN samples.retail.dim_date d ON f.date_key = d.date_key;
CREATE VIEW v_sales_top AS SELECT * FROM v_sales_enriched WHERE year = 2026;
```

⚠ **Offending pattern** Query the nested view:

```
/* ex11_view */ SELECT category, count(*) FROM v_sales_top GROUP BY category;
```

 **Find it** Compare EXPLAIN (ANALYZE) of the view query against the equivalent inlined SQL. Does the `year = 2026` filter push down to the `dim_date` scan (few date rows, fact bloom-filtered early), or does it run after the joins?

✓ **Fix** Inline (flatten) the view for the hot query path, and avoid expressions on view output columns so filters can still push down.

## Exercise 12 — Planner Hinting: Raising `join_collapse_limit`

Time: ~6 minutes

● **Concept** With many joins the planner stops permuting after 8 tables. For a heavy multi-join statement, raise `join_collapse_limit` so it can consider more orders — at the cost of longer planning, so use it where it pays off.

⚠ **Offending pattern** A star join written across all 7 dimensions. Run it once at the default limit (8) and once at 12, and read each plan:

```
/* ex12 */ EXPLAIN (ANALYZE) SETTING (join_collapse_limit = 12)
SELECT d.year, p.category, s.state, e.employee_key, pm.payment_method_key,
       t.time_key, c.membership_tier, count(*)
FROM samples.retail.fact_sales f
JOIN samples.retail.dim_date d           ON f.date_key = d.date_key
JOIN samples.retail.dim_product p        ON f.product_key = p.product_key
JOIN samples.retail.dim_customer c       ON f.customer_key = c.customer_key
JOIN samples.retail.dim_store s          ON f.store_key = s.store_key
JOIN samples.retail.dim_employee e       ON f.employee_key = e.employee_key
JOIN samples.retail.dim_payment_method pm ON f.payment_method_key = pm.payment_method_key
JOIN samples.retail.dim_time t           ON f.time_key = t.time_key
GROUP BY d.year, p.category, s.state, e.employee_key, pm.payment_method_key,
         t.time_key, c.membership_tier;
```

▶ **Find it** The plan footer echoes `join_collapse_limit 12`. Notice the planner **eliminates three of the joins** (`dim_employee`, `dim_payment_method`, `dim_time`): because the query only references their key columns — which also live on the fact table — the foreign keys let it skip those joins entirely. The four remaining dimensions are all broadcast (`DISTRIBUTE REPLICATE`). To force a join to stay, reference a **non-key** attribute of that dimension (e.g. `dim_store.state`).

🕒 **What you should see** At 351M rows this is the heaviest query in the module: it scans the fact, broadcasts the surviving dimensions, and the wide GROUP BY produces ~4.6M groups — Read 3.94 GiB, Distributed 245.94 MiB, ~22.7 s. Raising the limit lets the planner consider more orders but adds planning time, so apply it only to genuinely wide statements and confirm the plan actually improved first.

## Cleanup — Run at the End

Drop everything you created so your database stays tidy:

```
DROP TABLE IF EXISTS my_fact_dt, my_txn, my_sales_bad, my_sales_good, my_feed,
                     my_fact_bystore, my_sales, my_changes, my_sales_new, my_fact_skew,
                     store_rev_rs;
DROP VIEW IF EXISTS v_sales_top, v_sales_enriched;
DROP PROCEDURE IF EXISTS store_rev();
DROP PROCEDURE IF EXISTS store_rev(INT);
```

```
-- Keep qstat_rs + show_query_stats() if you want the stats helper for later modules.
```

## Quick Reference

---

### Anti-pattern → Fix

| You see this in the plan                                                    | Do this instead                                                                                                                 |
|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| A cast/function on the scanned column;<br>rows_planned far from rows_actual | Compare the bare column to a constant or<br>constant-side range                                                                 |
| DISTRIBUTE ON HASH(col) under a join or GROUP<br>BY                         | Distribute both tables on the join/group key; group<br>on the distribution column                                               |
| ::INT8 / calculate: in the ON clause                                        | Match key types; materialize expressions as<br>stored columns                                                                   |
| A small dimension as BUILD with DISTRIBUTE<br>REPLICATE                     | Already optimal — broadcast is cheap; declare<br>small dims REPLICATE in DDL                                                    |
| Three SCAN nodes for three COUNT(DISTINCT)                                  | Already near-optimal on columnar (each reads<br>only its columns); compute fewer distincts — do<br>NOT materialize a base table |
| A subquery/CTE with a bad estimate, recomputed                              | CTAS/TEMP table with the right DISTRIBUTE key +<br>ANALYZE                                                                      |
| High skew; one worker rows_actual >> others                                 | Split NULL/heavy keys out of the join (UNION ALL)                                                                               |

### The EXPLAIN (ANALYZE) metrics that matter

rows\_planned vs rows\_actual — estimate quality; a big gap mis-plans everything downstream.

read\_efficiency — how much of what you read you actually needed. DISTRIBUTE ON HASH(...) — data

moving between workers. skew — work balance across workers (0 is even). io\_spill\_space\_bytes\_max —

memory pressure spilling to disk.